PEARLS AQI PREDICTOR

A Multi-City Air Quality Forecasting System for Pakistan

FINAL INTERNSHIP PROJECT REPORT

pearls-aqi-predictor-intern.streamlit.app

Intern Name	Abdul Haseeb
Repository	github.com/Haseeb-py/pearls-aqi-predictor
Live Dashboard	pearls-aqi-predictor-intern.streamlit.app
Cities covered	Lahore, Karachi, Islamabad, Peshawar, Quetta, Sargodha
Forecast horizons	24 hours, 48 hours, 72 hours
Core stack	Python, scikit-learn, PyTorch, SHAP, Hopsworks, FastAPI, Streamlit, Groq
Report date	28 August 2026

. Table of Contents

1. Introduction

1.1 Background

Air pollution is a persistent and severe public health issue across Pakistan's major urban centres. Cities such as Lahore and Karachi regularly rank among the most polluted in the world during peak smog season, with the Air Quality Index (AQI) frequently crossing into the “Unhealthy” and “Hazardous” categories. Residents, schools, and health authorities currently have very limited access to forward-looking, city-specific air quality forecasts — most publicly available tools report only current conditions, not what is likely to happen over the next several days.

Pearls AQI Predictor was undertaken as an internship project to close part of that gap: to build a working, end-to-end machine learning system capable of forecasting AQI up to three days in advance for several major Pakistani cities, using a fully serverless technology stack so that the system carries no ongoing infrastructure cost or maintenance burden.

1.2 Problem Statement

Given historical weather and air-pollutant data for a city, predict its US Air Quality Index (AQI) at three future points in time — 24, 48, and 72 hours ahead — with sufficient accuracy to be genuinely more useful than naive assumptions such as “tomorrow will be the same as today.” The system must be reproducible across multiple cities, must not leak future information into its training process, and must expose its predictions through both a programmatic API and a human-facing dashboard.

1.3 Project Objectives

The original project brief specified the following required deliverables:

1. A feature pipeline that fetches raw weather and pollutant data from an external API and computes model-ready features, including time-based features (hour, day, month) and derived features such as AQI change rate.
2. A historical backfill process, running the feature pipeline across a range of past dates to generate a training dataset.
3. A training pipeline that fetches historical features and targets, trains and evaluates the best possible model for the data — experimenting with both classical models (Random Forest, Ridge Regression) and deep learning — and stores the result in a model registry.
4. An automated CI/CD pipeline that runs the feature script hourly and the training script daily, using a serverless orchestration tool.
5. A web application that loads the model and features, computes predictions, and displays them on a descriptive dashboard.
6. Supporting analysis: exploratory data analysis (EDA) to identify trends, SHAP or LIME for feature-importance explanations, and alerts for hazardous AQI levels.

Beyond this brief, the project was deliberately extended in four ways: (1) a multi-city selector covering six major Pakistani cities rather than one hardcoded city; (2) a natural-language “AQI Copilot” built on a tool-calling large language model, so that a non-technical user can ask questions in plain English and receive answers grounded in the real forecasting pipeline; (3) a live cloud deployment of both the backend API and the dashboard; and (4) a substantially deeper experimentation and validation process than the brief required, described in full in Section 7.

1.4 Scope and Non-Goals

The system forecasts AQI using modeled meteorological and atmospheric data (CAMS, via Open-Meteo); it does not use physical ground-station sensor readings, and this distinction is disclosed throughout the product rather than presented as ground-truth measurement. The project does not attempt to forecast beyond 72 hours, does not perform pollution-source attribution, and does not provide medical diagnoses — its health guidance is limited to standard, conservative US AQI category advisories.

2. System Architecture

The system is built as a batch-oriented, serverless pipeline rather than a continuously running service. Data is fetched and processed on a fixed schedule — hourly for feature updates, daily for model retraining — rather than streamed continuously. Section 9.1 explains this design choice and its trade-offs in detail.

2.1 End-to-End Pipeline

The system consists of eight sequential stages:

7. Data ingestion — raw weather and air-quality data retrieved from the Open-Meteo APIs for each configured city.
8. Validation — schema, range, and duplicate checks applied to incoming raw data before it is used.
9. Feature engineering — calendar, cyclical, lag, rolling, and pollutant/weather features computed for every hourly row; three forward-looking targets constructed (+24h, +48h, +72h).
10. Storage — engineered features persisted to local CSV artifacts (for offline development) and to the Hopsworks Feature Store (for cloud-based training and serving).
11. Model training and evaluation — multiple candidate models trained and benchmarked per city, per forecast horizon, using a strict chronological train/test split.
12. Model registry — the selected champion model per horizon, per city, persisted as a local joblib artifact and, for Lahore, registered in the Hopsworks Model Registry.
13. Serving — a FastAPI backend that loads the registered champion models and serves real-time predictions and Copilot responses.
14. Presentation — a Streamlit dashboard providing an overview, a cross-city comparison view, model analytics/explainability, and the AQI Copilot chat interface.

2.2 Technology Stack

Layer	Technology	Role
Language / runtime	Python 3.11	Primary implementation language across all pipelines and services
Data handling	pandas, NumPy	Tabular data processing, feature computation
Classical ML	scikit-learn	Ridge Regression and Random Forest implementations, preprocessing pipelines
Deep learning	PyTorch	Feed-forward neural network models, one per forecast horizon
Explainability	SHAP 0.51.0 (primary); permutation importance (fallback)	Global and local feature-importance explanations
Feature store / registry	Hopsworks 5.0.4	Cloud-hosted feature group, feature view, and model registry
Backend API	FastAPI + Uvicorn	REST endpoints for predictions and Copilot chat
Dashboard	Streamlit	Interactive web dashboard
Visualization	Matplotlib, Seaborn	Exploratory data analysis charts
LLM / Copilot	Groq API (llama-3.3-70b-versatile)	Tool-calling conversational assistant
Automation	GitHub Actions	Scheduled, serverless pipeline execution and CI
Deployment	FastAPI Cloud; Streamlit Community Cloud	Live hosting of the API and dashboard respectively

3. Data Sources and Collection

3.1 Provider and Endpoints

All data originates from Open-Meteo, a free weather and air-quality data provider. Two families of endpoints are used:

- **Air Quality API** (air-quality-api.open-meteo.com) — provides US AQI along with PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, and ozone concentrations, sourced from the CAMS (Copernicus Atmosphere Monitoring Service) global atmospheric model.
- **Weather Forecast and Historical Weather APIs** (api.open-meteo.com and archive-api.open-meteo.com) — provide temperature at 2 metres, relative humidity, surface pressure, wind speed, and precipitation, for both forecast and historical date ranges.

> **Data Provenance**
> This is modeled atmospheric data, produced by a numerical simulation, not measurements from physical ground-station sensors. This distinction is disclosed explicitly in the dashboard and API responses rather than presented as raw sensor truth, in line with responsible data-provenance practice.

3.2 Backfill Strategy

Historical data is required to train any supervised forecasting model. The project's backfill process re-runs the feature-generation logic across a specified historical date range, producing a labelled training dataset (features paired with the AQI value that actually occurred 24/48/72 hours later).

The backfill window used for the final, production models is three years per city — 2022-08-01 through 2025-08-01. This was not an arbitrary choice: Section 7.6 documents a controlled experiment specifically testing whether three years of data outperforms one year, using an identical held-out test period for both, to isolate data volume as the only variable under study.

3.3 Per-City Data Inventory

City	Backfill range	Rows collected	Model status
Lahore	2022-08-01 – 2025-08-01	26,328	Registered (local + Hopsworks Model Registry)
Karachi	2022-08-01 – 2025-08-01	26,328	Registered (local registry)
Islamabad	2022-08-01 – 2025-08-01	26,328	Registered (local registry)
Peshawar	2022-08-01 – 2025-08-01	26,328	Registered (local registry)
Quetta	2022-08-01 – 2025-08-01	26,328	Registered (local registry)
Sargodha	2022-08-01 – 2025-08-01	26,328	Registered (local registry)

Total stored three-year backfill across all six cities: 157,968 rows. Superseded Lahore-only extracts from earlier experimentation phases (one-year and thirty-day windows) remain on disk but are not used by any current champion model.

4. Feature Engineering

Raw API data alone is not sufficient input for a forecasting model. Every hourly row, for every city, is transformed into a richer feature vector designed to expose the temporal and physical patterns that drive AQI. This section documents every feature group actually implemented and the reasoning behind each.

4.1 Calendar and Cyclical Features

Hour of day, day of month, month, day of week, and a weekend flag are included because air pollution follows strong daily and seasonal rhythms — traffic-driven rush-hour peaks, weekday/weekend differences in industrial activity, and seasonal smog patterns. Hour-of-day and month are additionally encoded using sine and cosine transforms, so that, for example, hour 23 and hour 0 are represented as numerically adjacent (as they are in real time), rather than as maximally distant values, which a plain numeric encoding would incorrectly imply.

4.2 AQI Lag Features

The model receives the AQI value observed 1, 3, 6, 12, 24, 48, and 72 hours before the current row. These are the single strongest predictors in the feature set: air pollution changes gradually rather than randomly, and recent AQI levels are highly informative about near-term future levels. Their usefulness diminishes the further into the future a lag is compared against — an observation from one hour ago is far more informative for a 24-hour forecast than for a 72-hour forecast, which is a central reason forecast accuracy degrades with horizon (see Section 8.5).

4.3 Change-Rate and Rolling Features

Change-rate features (1h, 3h, and 24h AQI change, plus their percentage equivalents) capture whether pollution is currently rising, falling, or stable — information not present in a raw lag value alone. Rolling mean and standard deviation over 3, 6, 12, and 24-hour windows smooth out short-term noise (a single anomalous hourly reading) and expose the underlying trend. Rolling statistics are computed using a one-step shift before the rolling window is applied, ensuring the current hour's own value never contributes to its own rolling statistic.

4.4 Weather and Pollutant Features

Current temperature, relative humidity, surface pressure, wind speed, and precipitation are included as direct physical drivers of pollutant dispersion and accumulation. Current concentrations of PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, and ozone are included alongside the current AQI value itself, since AQI is a composite index and its constituent pollutants can behave differently under different conditions.

4.5 Forecast-Weather Features

For select 48-hour and 72-hour model variants, forecasted (rather than historical) weather fields from the Open-Meteo forecast endpoint are used as additional inputs, on the reasoning that a real deployment genuinely has access to a forward-looking weather forecast at prediction time. Section 7.2 documents this as a formally tested hypothesis, including the specific case where it did not generalise well to historical backfill data and was reverted for the affected models.

4.6 Complete Feature Inventory

Category	Features
Calendar	hour, day, month, day of week, weekend flag
Cyclical encoding	sine/cosine of hour; sine/cosine of month
Current weather	temperature, relative humidity, surface pressure, wind speed, precipitation
Pollutants	PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, ozone, current US AQI
AQI lags	1h, 3h, 6h, 12h, 24h, 48h, 72h
AQI change / rate	change rate and percent change at 1h, 3h, 24h
Rolling statistics	AQI mean and standard deviation at 3h, 6h, 12h, 24h windows
Forecast weather	Open-Meteo forecast fields, used in select 48h/72h model variants
Targets (labels)	AQI at +24h, +48h, +72h — three independent supervised targets

4.7 Leakage Prevention

A recurring and serious risk in time-series machine learning is data leakage — accidentally allowing a model to see information from the future during training, which produces misleadingly optimistic evaluation results that do not hold up in real deployment. This was treated as a first-class engineering concern:

- Data is sorted by city and UTC timestamp before any lag or target computation is performed.
- Lag features use strictly positive (backward-looking) time shifts, computed independently per city.
- Rolling windows apply a one-step shift before the rolling calculation, so the current hour never contributes to its own rolling statistic.
- Target columns use negative (forward-looking) time shifts, computed independently per city.
- A temporal leakage assertion programmatically rejects any row whose feature timestamp falls after a supplied cutoff, as an automated safety check rather than relying solely on correct implementation.

Implementation Note

An early version of the pipeline computed lag and rolling features independently within each backfill batch, rather than across the full continuous city history. Because a single batch rarely contained 72 hours of prior context, this caused nearly every lag column to be null immediately after a batch boundary, which in turn caused the training set to collapse to zero usable rows. The fix, now standard in the training data loader, reassembles all available history for a city, de-duplicates it, sorts it chronologically, and recomputes every lag, rolling, and target feature across the complete continuous series before any model sees it.

5. Modeling Methodology

5.1 Chronological Train/Test Split

All models are evaluated using a chronological split: the earliest approximately 80% of each city's data is used for training, and the most recent approximately 20% is held out for testing. This is deliberately different from a random split. A random split would allow the model to be trained on data chronologically after some of its test examples — effectively letting it “see the future” relative to part of the test set — which would produce unrealistically optimistic accuracy figures that would not hold up when the model is actually used to forecast forward in time, as it always will be in production.

5.2 Evaluation Metrics, Explained

Three complementary metrics are reported for every model and every horizon:

Mean Absolute Error (MAE)

The average absolute size of the model's prediction error, in AQI points. It is calculated by taking the absolute difference between each prediction and the true value, then averaging across all test examples. An MAE of 27 means that, on average, the model's prediction is off by about 27 AQI points in either direction. Lower is better; zero represents perfect prediction.

Root Mean Squared Error (RMSE)

Similar to MAE, but each error is squared before averaging and then the square root is taken at the end. Squaring disproportionately penalises large errors — an error of 50 contributes 25 times more to RMSE than an error of 10, not simply 5 times more. When RMSE is substantially higher than MAE for the same model, that gap indicates the presence of occasional large outlier mistakes rather than uniformly moderate error.

R^2 (Coefficient of Determination)

Unlike MAE and RMSE, R^2 is not a raw error measurement but a relative comparison against the simplest possible baseline: always predicting the mean value of the test period, regardless of input. An R^2 of 1.0 indicates perfect prediction. An R^2 of 0.0 indicates the model performs exactly as well as blindly guessing the average — it has learned nothing useful. A negative R^2 indicates the model performs worse than that trivial baseline, which is a meaningful warning sign rather than merely “weak” performance.

5.3 Candidate Models

Model	Description	Rationale for inclusion
Persistence baseline	Predicts that future AQI equals the current AQI, unchanged.	Sanity-check floor: any real model must beat this to be considered useful.
Seasonal-naive baseline	Predicts that future AQI equals the AQI observed at the same hour 24 hours earlier.	Tests whether the model has learned anything beyond simple daily repetition.
Ridge Regression	Regularized linear regression with median imputation and feature scaling.	Handles many correlated features smoothly; generalizes gracefully to unfamiliar conditions; fast and interpretable.
Random Forest	An ensemble of many decision trees, averaged; hyperparameters tuned per horizon.	Captures non-linear relationships and interactions between weather, pollutant, and time features.
PyTorch neural network	A compact feed-forward network, trained independently per horizon.	Satisfies the project's deep-learning experimentation requirement; capable of learning complex non-linear patterns given sufficient data.

Both baselines exist specifically to keep the real models honest. Early testing showed both baselines scoring negative R^2 at almost every horizon — meaning Lahore's actual AQI behaviour does not reliably follow either “stays the same” or “repeats yesterday.” This made the subsequently positive R^2 achieved by the real models a genuine, non-trivial result rather than an easy target.

5.4 Per-Horizon, Per-City Model Training

Because the relationship between the input features and the target genuinely changes depending on how far ahead the forecast is (see Section 8.5), each model type is trained separately for each of the three horizons, rather than one model attempting to serve all three. Concretely, this means each city has three independently trained Ridge models, three independently trained Random Forest models, and three independently trained neural networks — fifteen distinct trained predictors per city when baselines are included — each evaluated on its own horizon-specific test set.

6. Experimentation and Model Refinement

Lahore was used as a vertical-slice proving ground: the complete methodology below — baseline comparison, refinement experiments, and per-horizon champion selection — was developed and validated on Lahore first, and then independently repeated, not copied, for each of the remaining five cities, since different cities plausibly have different optimal models given different local pollution and weather dynamics (confirmed in Section 8, where the champion model type varies by city).

6.1 Motivation

Initial baseline testing showed that while 24-hour forecasts were immediately workable ($R^2 \approx 0.27$), 48-hour and especially 72-hour forecasts were poor, in the worst case (Random Forest at 72 hours) scoring $R^2 = -9.47$ — dramatically worse than simply guessing the average. Rather than accept this, six further experiments were designed, each testing one specific, falsifiable hypothesis about why longer-horizon performance was weak, and each measured rigorously before being kept or discarded.

6.2 Experiment Log

#	Experiment	Hypothesis	Outcome	Disposition
1	Baseline 5-model comparison	Establish a reference point across all model types and horizons.	24h workable ($R^2 \approx 0.27$); 48h/72h weak; RF 72h $R^2 = -9.47$.	Diagnostic baseline
2	Forecasted-weather features (approximated from historical data for backfill dates)	Future weather information should help longer-horizon predictions.	Made Ridge and Random Forest worse at 48h/72h (Ridge 72h R^2 fell to −2.06).	Reverted
3	Feature pruning (removed redundant/correlated lag & rolling columns)	Fewer, cleaner inputs should reduce noise, especially for the neural network.	Neural network 48h/72h improved substantially (72h R^2: −0.155 → −0.059).	Kept — Pruned Neural Network
4	Stronger Ridge regularization	Ridge is overfitting at longer horizons; a stronger penalty should help.	Ridge 48h/72h improved substantially (72h R^2: −2.06 → −0.606).	Kept — Strong-Regularized Ridge
5	Validation-weighted Ridge/RF blend	An optimal weighted mix of Ridge and Random Forest may beat either alone.	Best-ever 24h result ($R^2 = 0.281$); correctly assigned 0% Ridge weight at 48h/72h.	Kept for 24h only
6	Per-horizon champion selection (vs. one champion for all horizons)	Different horizons need different model complexity, not one universal winner.	Worst-case R^2 improved from −9.47 to −0.059.	Kept — adopted as standard
7	One-year vs. three-year backfill, re-verified on an identical holdout window	More historical training data should improve accuracy, especially at longer horizons.	R^2 improved at every horizon; 72h gained the most (+0.181).	Kept — standardized to 3 years

6.3 The Reverted Experiment, in Detail

Experiment 2 is worth examining closely because it demonstrates the value of testing each change in isolation rather than accepting a plausible-sounding idea on its own merits. In a real deployment, a genuine weather forecast for a future timestamp is available at prediction time, so using forecasted temperature, humidity, wind, and pressure as model inputs for the 48h/72h targets is a reasonable idea in principle. However, Open-Meteo's forecast API only covers the following approximately 16 days from the current date — there is no way to retrieve "what the forecast would have said" for a date a year or more in the past. The historical backfill therefore had to approximate this feature using historical (not forecast) weather values, and that approximation introduced noise rather than genuine forward-looking signal, measurably degrading Ridge and Random Forest performance at the horizons it was meant to help.

6.4 Controlled Verification: Does More Data Really Help?

The initial one-year-vs-three-year comparison used two different chronological test windows (each dataset's own final 20%), which meant any observed improvement could not be safely attributed to data volume alone — it might equally have reflected the three-year run's test period simply covering a more predictable stretch of real-world weather. To isolate training data volume as the only variable, a controlled re-test was performed: the one-year model was re-evaluated on the exact same holdout window as the three-year model (2024-12-25 through 2025-08-01, 5,266 shared test rows).

Horizon	Model	1-yr MAE	3-yr MAE	1-yr R^2	3-yr R^2	R^2 change
24h	Ridge/RF blend	24.61	22.96	0.500	0.521	+0.020
48h	Pruned neural network	33.59	32.43	0.133	0.149	+0.016
72h	Pruned neural network	36.74	32.17	0.013	0.195	+0.181

With training volume the only variable changed, and both models judged on identical unseen data, three years of history improved every horizon — with the largest and most valuable gain precisely at the horizon that had been weakest throughout the project. This finding justified standardizing all six cities on a three-year backfill for their production models.

7. Final Results — All Cities

The table below is the complete, current set of per-horizon champion models and their held-out evaluation metrics for all six supported cities, drawn from each city's registered model metadata. R^2 values are colour-coded: green (≥ 0.4, strong signal), amber (0–0.4, positive but modest signal), red (negative, weaker than a naive average-guessing baseline).

City	Horizon	Champion model	MAE	RMSE	R^2
Lahore	24h	Ridge / RF blend (5% / 95%)	22.96	37.16	**0.521**
Lahore	48h	Pruned neural network	32.43	49.10	**0.149**
Lahore	72h	Pruned neural network	32.17	47.67	**0.195**
Karachi	24h	Ridge / RF blend (15% / 85%)	10.51	14.62	**0.545**
Karachi	48h	Ridge (alpha 1000)	13.97	18.62	**0.243**
Karachi	72h	Ridge (alpha 1000)	15.25	20.16	**0.113**
Islamabad	24h	Pruned neural network	13.93	18.26	**0.698**
Islamabad	48h	Ridge (alpha 300)	19.28	24.67	**0.445**
Islamabad	72h	Ridge (alpha 100)	20.19	25.99	**0.378**
Peshawar	24h	Pruned neural network	14.46	18.86	**0.647**
Peshawar	48h	Ridge / RF blend (55% / 45%)	18.18	23.51	**0.450**
Peshawar	72h	Ridge (alpha 100)	19.19	24.67	**0.391**
Quetta	24h	Ridge (alpha 30)	15.96	26.14	**0.434**
Quetta	48h	Pruned Ridge (alpha 30)	20.77	32.60	**0.122**
Quetta	72h	Pruned Ridge (alpha 30)	22.44	34.79	**0.002**
Sargodha	24h	Ridge / RF blend (90% / 10%)	19.65	26.10	**0.687**
Sargodha	48h	Ridge / RF blend (50% / 50%)	26.95	34.61	**0.432**
Sargodha	72h	Ridge (alpha 1000)	28.58	36.37	**0.352**

7.1 Interpretation by Horizon

- 24-hour forecasts are strong and consistent across every city, ranging from $R^2 = 0.434$ (Quetta) to $R^2 = 0.698$ (Islamabad) — in every case a substantial, genuine improvement over guessing the average, and the most immediately actionable forecast for a real user.
- 48-hour forecasts are positive for every city, ranging from $R^2 = 0.122$ (Quetta) to $R^2 = 0.450$ (Peshawar).
- 72-hour forecasts are positive across the supported cities, ranging up to $R^2 = 0.391$ (Peshawar), with Quetta's longer-horizon result reflecting its distinct local pollution and weather profile.

7.2 Interpretation by Model Type

No single algorithm wins everywhere: pruned neural networks, plain Ridge, regularized Ridge, and Ridge/Random-Forest blends each appear as the selected champion for at least one city/horizon combination. This is itself evidence supporting the per-horizon, per-city selection architecture described in Section 5.4 — forcing a single “best” algorithm onto every city and horizon would have left value on the table for most of them.

7.3 Why Accuracy Degrades With Forecast Horizon

Across every city, 24-hour forecasts outperform 48- and 72-hour forecasts by a wide margin. This is an expected, well-documented property of time-series forecasting generally — comparable to why next-day weather forecasts are far more reliable than five-day forecasts — and is not a defect isolated to this project's models. Three specific, evidenced reasons support this interpretation:

- The model's strongest features are AQI lags. A one-hour-old AQI reading is highly informative for a 24-hour forecast, but by 72 hours ahead, substantially more time has passed for genuinely new events (a wind shift, an unexpected rain event, a new pollution episode) to occur that no lag feature could have anticipated.
- More training history helped most at the longest horizon (Section 6.4, +0.181 R^2 at 72h vs. +0.020 at 24h) — consistent with a longer horizon needing more historical examples of rare, hard-to-predict events to generalize well.
- Forward-looking weather features were hypothesized specifically to compensate for this effect (Section 6.3); the properly-implemented, real-time version of that feature (available for live predictions, as opposed to the historical approximation used in backfill) remains a promising direction not yet fully exploited, and is listed as future work in Section 16.

8. Feature Store and Model Registry

Hopsworks is used as the project's cloud-hosted feature store and model registry, operating alongside a local-artifact fallback that keeps the system fully functional for offline development and resilient to transient cloud unavailability.

8.1 Hopsworks Resources

Resource	Name
Raw observation feature group	aqi_observations_v1
Engineered feature group	aqi_features_hourly_v2
Feature view	aqi_features_fv_v2
Model naming pattern	pearls_aqi_predictor_<city>

An earlier feature group version (v1) was created during initial testing with a minimal schema and became incompatible once the full engineered feature set was finalized; a clean v2 feature group and matching feature view were created to resolve the schema mismatch, and this is the version in current use.

8.2 Model Registry Status

Model Registry versions confirmed in the Hopsworks console: Lahore is at version 7, reflecting multiple re-registrations across the experimentation process documented in Section 6; Karachi, Islamabad, Peshawar, Quetta, and Sargodha are each at version 1, representing their first registration under the now-standardized three-year methodology.

> **Implementation Note**
> The Hopsworks SDK's model-upload function moves local model files by default rather than copying them, which would have silently broken the local registry immediately after the first cloud upload. This was diagnosed and fixed by explicitly passing keep_original_files=True to the upload call, and the local artifact was verified to remain fully intact afterward.

9. Key Design Decisions and Rationale

9.1 Batch Processing, Not Streaming

The system fetches and processes data on an hourly/daily schedule rather than continuously streaming it. Batch processing collects data over a period and processes it together on a fixed schedule, analogous to running one load of laundry per day rather than washing each item the instant it becomes dirty; streaming processes each event the moment it arrives, at the cost of substantially more infrastructure (message queues, always-on stream processors). Batch processing is the appropriate choice here because AQI changes meaningfully over hours, not seconds, and because the underlying data source itself (Open-Meteo) only provides hourly granularity — streaming infrastructure would add real cost and complexity without a corresponding accuracy or usefulness benefit. The explicit trade-off, disclosed to the user via a stale-data indicator in both the API and dashboard, is that displayed data can be up to approximately one hour old under normal scheduled operation.

9.2 Per-Horizon Champion Models

Rather than selecting one “best overall” model to serve all three forecast horizons, each horizon is served by its own independently validated champion. Section 6.2 (experiment 6) demonstrated this reduced the project's worst-case error from $R^2 = -9.47$ to $R^2 = -0.059$ — a result not achievable by any single-model approach, since the best model for a 24-hour problem and the best model for a 72-hour problem are demonstrably different (Section 7.2).

9.3 GitHub Actions Over Jenkins or Apache Airflow

The project brief specifically required a serverless stack. Jenkins and Apache Airflow are both capable, industry-standard orchestration tools, but both require a continuously running server or hosted scheduler process to operate — infrastructure that would have to be provisioned, patched, and paid for even while idle, which directly conflicts with the serverless requirement. GitHub Actions provisions temporary, on-demand runners that exist only for the duration of a scheduled job and are destroyed immediately afterward, which matches the serverless requirement exactly and required no server management on the project's part.

9.4 SHAP With a Permutation-Importance Fallback

The project brief specifically named SHAP or LIME for feature-importance explanations. SHAP was initially unavailable in the development environment, so permutation importance — a technique already built into scikit-learn, requiring no additional dependency — was used as an interim, model-agnostic substitute. SHAP was subsequently installed and integrated as the primary explainability method, satisfying the original requirement directly, with permutation importance retained as an automatic fallback so the explainability feature degrades gracefully rather than failing outright should SHAP become incompatible with a future model type.

9.5 A Grounded Copilot, Not a General-Purpose Chatbot

The AQI Copilot is deliberately restricted to seven allow-listed tools, each mapped directly to a real pipeline function, and is explicitly instructed never to state a number that did not originate from a tool call within the current conversation. This is the practical difference between a genuinely trustworthy assistant and a demo that occasionally states a confident but fabricated AQI value — the latter would be actively misleading in a health-relevant application, and grounding was treated as a non-negotiable requirement rather than a nice-to-have.

10. Explainability

Understanding why a model produced a given prediction is important both for trust (a user should be able to ask “why is tomorrow's forecast high?” and receive a real answer) and for debugging. The project implements two complementary layers of explainability.

10.1 SHAP (Primary Method)

SHAP (SHapley Additive exPlanations) is a library based on a game-theory concept (Shapley values) that fairly attributes a model's output to each of its input features, both globally (which features matter most overall) and locally (for one specific prediction, exactly how much did each feature push the result up or down). Two functions are implemented:

- shap_feature_importance() — uses SHAP's TreeExplainer for tree-based models (Random Forest) and LinearExplainer for linear models (Ridge), producing a global ranking of feature influence.
- shap_local_explanation() — explains one specific prediction in terms of each feature's individual contribution, e.g. identifying that elevated PM2.5 and low wind speed are the primary drivers of a particular high forecast.

10.2 Permutation Importance (Fallback Method)

Permutation importance works by measuring how much a trained model's accuracy degrades when one input feature's values are randomly shuffled, breaking its real relationship with the outcome while leaving every other feature untouched. A feature whose shuffling causes a large accuracy drop is judged important; one whose shuffling barely changes accuracy is judged unimportant. This technique is already built into scikit-learn and requires no additional dependency, making it a reliable, dependency-light fallback whenever SHAP is unavailable or incompatible with a given model.

The Streamlit Model Analytics tab surfaces both global feature importance and local, per-prediction explanations, and the same explain_prediction capability is exposed as a Copilot tool, so a user can ask in natural language why a given forecast came out the way it did and receive an answer grounded in the model's actual internal reasoning rather than a generic explanation.

11. Backend API

11.1 Endpoints

Endpoint	Purpose
GET /cities	Lists the enabled cities and their configuration
GET /predict/{city}	Returns current AQI plus 24h/48h/72h forecasts, AQI category, hazard flag, staleness flag, and model version
POST /api/v1/copilot/chat	Grounded, tool-using AQI Copilot conversational endpoint

11.2 Response Content

A successful call to /predict/{city} returns: the city identifier, the latest observation timestamp, the current AQI value, the AQI standard and data-provenance label, the model name and version, the 24h/48h/72h forecasts, the AQI category and hazardous-status flag for each horizon, and a stale-data flag indicating whether the underlying observation is older than the expected freshness window.

11.3 Reliability Behavior

- Unknown or untrained cities return a clean HTTP 404 response rather than crashing or silently returning a guessed value.
- Downstream failures — registry errors, missing feature data, or weather-provider errors — return HTTP 503 rather than an unhandled server error.
- Application startup does not require Hopsworks or Groq connectivity; cloud calls are made lazily, only when a specific request actually needs them, and startup itself catches cache warm-up failures so the API remains available even if a dependency is briefly unreachable.

12. Dashboard

Tab	Contents
Overview	City selector; current/24h/48h/72h AQI cards; historical and forecast trend chart; pollutant snapshot; hazardous-AQI and stale-data alerts
City Comparison	Side-by-side current and 24h/48h/72h forecasts across all trained cities, presented as both a table and a bar chart
Model Analytics	Saved evaluation metrics; the current champion selection per horizon; global SHAP/permutation feature importance; local (per-prediction) explanations
AQI Copilot	Conversational interface: message history, suggested example questions, a text input, and grounded answers with data-freshness metadata

Hazardous-AQI alerting uses the standard US AQI category breakpoints and displays a visible warning whenever any forecasted horizon reaches “Unhealthy” (AQI ≥ 151) or worse, covering the full standard scale from “Good” (0–50) through “Hazardous” (301+).

13. The AQI Copilot

The Copilot is a grounded, tool-calling conversational assistant, built to let a non-technical user ask natural-language questions and receive answers backed by the real forecasting pipeline — never by the language model's own general training knowledge. This design directly addresses the central risk of LLM-based assistants in a health-relevant context: confident, plausible-sounding, but fabricated answers.

Property	Value
Default enabled state	COPILOT_ENABLED = False (opt-in; deferred until the core forecasting product was stable)
LLM provider	Groq
Model	llama-3.3-70b-versatile
Conversation history window	Last 6 messages
Logging	Tool name, outcome, latency, and correlation ID recorded per tool call

13.1 Tools Available to the Copilot

The Copilot can only answer using seven explicitly allow-listed tools, each mapped directly to a real function in the pipeline: get_current_aqi, get_aqi_forecast, get_weather, get_pollutants, compare_cities, get_aqi_history, and explain_prediction. It has no ability to answer outside of what these tools can return.

13.2 Verified Safety and Quality Behavior

- Refuses to invent an AQI value for unsupported cities and states plainly that no model exists for them.
- Distinguishes current, historical, and forecasted values in every response, and explicitly flags stale stored data rather than presenting it as live.
- Resists prompt-injection attempts to override its instructions, reveal its internal system prompt or tool list, or fabricate a specific requested number.
- Correctly answers multi-part natural-language questions (for example, a single message combining a status question, an action question, and a general-knowledge question) following an iteration that fixed an earlier version which only addressed the first part of such a message.
- Gives appropriately tiered health guidance (e.g. for outdoor exercise) distinguishing sensitive groups from the general population, without overreaching into unqualified medical advice.

14. Automation

Workflow	Schedule	Purpose
Hourly feature pipeline	17 (hourly)	Runs the feature pipeline for all configured cities
Daily model training	35 1 (daily)	Retrains and re-registers champion models for all supported cities
CI	On push and pull request	Installs dependencies; runs Ruff linting and the pytest suite with coverage

All scheduled workflows include manual dispatch triggers, for on-demand testing, and concurrency guards, to prevent overlapping runs of the same job.

15. Testing and Quality Assurance

The project maintains a pytest suite spanning three tiers: unit tests (fast, fully offline), contract tests (validating the real shape of Open-Meteo API responses), and integration tests (real Hopsworks/Open-Meteo network calls, skipped by default and run only via an explicit flag, to keep the default test run fast and independent of external service availability).

Most recent full run: 38 passed, 5 skipped, in 56.6 seconds. The 5 skipped tests are the opt-in, live-service integration tests.

Test coverage specifically includes: feature and target leakage assertions; chronological split correctness; API error handling (404 for unknown/untrained cities, 503 for downstream failures); hazardous-AQI alert threshold behavior; and Copilot tool-selection accuracy and prompt-injection resistance.

16. Future Enhancements

- Extend Quetta's longer-horizon modeling with city-specific feature engineering tuned to its local pollution and weather profile.
- Activate and monitor the hourly/daily GitHub Actions workflows on a continuous live schedule against production secrets.
- Persist per-horizon training-row counts and full experiment metrics alongside champion metadata for streamlined historical auditing.
- Extend Hopsworks Feature Store and Model Registry integration to all six cities, matching Lahore's current cloud-registered status.
- Align the Copilot's data path with the main API's enriched feature set so 48h/72h explanations are consistently available.
- Expand SHAP explanation coverage across every city, model type, and horizon combination in the live deployment.
- Integrate a genuine real-time forecast-weather feature into live serving to further improve 48h/72h accuracy.
- Add production monitoring and drift detection to track model performance over time as new data accumulates.

17. Glossary of Terms

Term	Definition
AQI	Air Quality Index — a standardized scale summarizing pollutant concentrations into a single number and category (e.g. Good, Moderate, Unhealthy).
Batch processing	Collecting and processing data on a fixed schedule, rather than continuously as it arrives (contrast: streaming).
Champion model	The single best-performing model selected for production use, for a given city and forecast horizon.
Chronological split	Dividing data into training and test sets by time, so the model is always tested on data later than what it trained on.
Feature	One input variable given to a model, e.g. temperature or AQI 24 hours ago.
Feature store	A managed storage system for engineered features, enabling consistent reuse across training and serving.
Grounding	Restricting an LLM's answers to information retrieved from real tools/data, rather than its own generated knowledge.
Horizon	How far into the future a forecast is made — this project uses 24h, 48h, and 72h horizons.
Lag feature	The value of a variable at a previous point in time, e.g. AQI 24 hours ago.
MAE	Mean Absolute Error — the average size of a model's prediction error, in the original units.
Model registry	A versioned storage system for trained models and their metadata.
R^2	A relative measure of model skill versus predicting the average value; 0 = no better than average, negative = worse than average.
RMSE	Root Mean Squared Error — similar to MAE, but penalizes large errors more heavily.
Rolling statistic	A moving average or standard deviation computed over a recent time window.
SHAP	SHapley Additive exPlanations — a library for attributing a model's prediction to its individual input features.
Tool calling	An LLM capability allowing it to invoke external functions/APIs to retrieve real data rather than generating an answer from memory.

18. Conclusion

Pearls AQI Predictor demonstrates a complete, evidence-driven machine learning system: real multi-city data ingestion, leakage-safe feature engineering, a rigorously benchmarked set of classical and deep-learning models, a fully documented experimentation trail including both successful and reverted changes, cloud storage and registry integration, a live API and dashboard, and a grounded conversational assistant. Its 24-hour forecasts are reliably useful across all six supported cities; its longer-horizon forecasts are honestly reported, including the one case — Quetta at 72 hours — where the model does not yet outperform a naive baseline.

The project's strongest evidence of engineering maturity is not any single metric, but the process documented in Section 6: hypotheses were proposed, tested in isolation, measured against a fair and controlled comparison, and kept or discarded strictly on the basis of results — including one experiment, forecasted-weather features, that was reverted after it measurably made results worse. That discipline, applied consistently from the first baseline comparison through the final six-city rollout, is the project's core deliverable, alongside the working forecasting system, API, dashboard, and Copilot themselves.